

FPGA-Based Lightweight FOD Classification

CNN Feature Extraction Project — Complete Project Recap

Absolutely. We've reached a **very good milestone** in your lightweight FOD classification project. Here's the complete recap from the beginning to where we are now.

1. Original objective

Your original project was:

****FPGA-based lightweight CNN/ML system for real-time FOD classification on a PYNQ-Z2.****

Initially, you were using **128×128 grayscale images** and a CNN.

The problem was that the CNN itself was relatively expensive for the PYNQ-Z2, so we moved toward a much lighter architecture:

```
RGB Image
↓
Feature extraction on ARM/PS
↓
64 features
↓
INT8 quantization
↓
Small neural network on FPGA/PL
↓
4-class prediction
```

The inspiration was the radar FPGA classifier architecture you found.

2. We changed from grayscale to RGB

Your earlier grayscale system had poor classification of the fuelcap class.

You had a result around:

```
Overall accuracy ≈ 87-88%
Fuelcap accuracy = 0%
```

We realized that converting:

RGB → Grayscale

throws away potentially useful color information.

So we decided to return to **RGB images**, while still keeping the feature vector small.

The important idea was:

128\times128\times3

\rightarrow

64\textrm{ features}

rather than:

64\times3=192\textrm{ features}

We wanted **64 total features**, regardless of the three RGB channels.

3. We adopted the more sophisticated feature extractor

You had a set of Python files generated from the Claude/reference implementation:

```
fod_features.py
fod_quant.py
build_dataset.py
check_masks.py
```

Instead of our earlier simple RGB extractor, we decided to use this more sophisticated object-centric feature extraction approach.

The feature extractor essentially performs:

```
RGB image
↓
Resize
↓
CIE Lab representation
↓
Background estimation
↓
Anomaly map
↓
Object segmentation
↓
Feature extraction
↓
64 features
```

The 64 features are divided into groups involving:

- object colour
- contrast relative to background
- shape
- gradients
- texture
- background context

This is particularly suitable for your FOD problem because the **object is being compared with the runway/background**, rather than simply looking at raw pixels.

4. We fixed an OpenCV compatibility problem

When we first tested `fod_features.py`, we got:

```
IndexError:
too many indices for array:
array is 2-dimensional, but 3 were indexed
```

The problem was in:

```
defects[:, 0, 3]
```

from the convexity-defect calculation.

We modified that section to reshape the OpenCV output:

```
defects = defects.reshape(-1, 4)
```

and then use:

```
defects[:, 3]
```

After correcting this, the extractor worked.

5. We tested one RGB image

We tested:

```
fuelcap/image_00077.png
```

and got:

```
Shape: (64,)
Min: -10.594955
Max: 57.051193
Object found: True
Mask area: 1263
```

More importantly, we visualized:

```
Original image
  ↓
Anomaly map
  ↓
Object mask
```

and the mask successfully isolated the fuelcap.

That was an important validation step because the shape/texture features depend on the segmentation being meaningful.

6. We processed the entire dataset

The current dataset is **not the earlier fuelcap/pliers/rock/screwdriver dataset**.

Your current RGB dataset contains:

```
fuelcap
pliers
screwdrivers
sodacan
```

The training set was successfully processed:

```
fuelcap      301
pliers       597
screwdrivers  428
sodacan      544
-----
Total        1870
```

And:

```
Errors:      0
No object:   0
```

So every training image successfully produced a 64-feature vector.

7. Dataset sizes

We processed all three splits.

Training

$X_{\text{train}}=(1870,64)$

$y_{\text{train}}=(1870,)$

Validation

$X_{\text{val}}=(384,64)$

$y_{\text{val}}=(384,)$

Test

$X_{\text{test}}=(412,64)$

```
y_{test}=(412,)
```

Therefore:

```
\boxed{2666\text{ images}}
```

were processed into:

```
\boxed{64\text{-feature vectors}}
```

8. We standardized the features

The raw 64 features had very different numerical ranges.

So we fitted a `FeatureScaler` **using only the training set**.

It produced:

```
Mean shape: (64,)
Std shape:  (64,)
Clip:       4.0
```

The standardized training data became approximately:

```
Min  = -4
Max  = +4
Mean ≈ 0
Std  ≈ 1
```

This is important because we don't want features with large numerical ranges dominating the small neural network.

And using only the training set to calculate the statistics prevents validation/test information from leaking into preprocessing.

9. We quantized the features to INT8

The standardized features were then converted:

`FP32` $\rightarrow$ `INT8`

with approximately:

$-127 \leq x \leq 127$

The resulting datasets are:

```
X_train_int8 → (1870, 64)
X_val_int8   → (384, 64)
X_test_int8  → (412, 64)
```

with:

```
dtype = int8
range = -127 ... +127
```

This is very important for your FPGA architecture because the FPGA doesn't need to receive floating-point values.

10. The FPGA data-transfer concept

With 64 INT8 features:

$64 \times 8 = 512$ bits

We can pack:

$4 \times \text{INT8} = 32$ bits

Therefore:

$64 / 4 = 16$

32-bit words are required to transfer the complete feature vector.

So the intended hardware interface becomes approximately:

```
64 INT8 features
  ↓
4 features / 32-bit word
  ↓
16 × 32-bit words
  ↓
AXI
  ↓
FPGA
```

This matches the architecture you were trying to replicate.

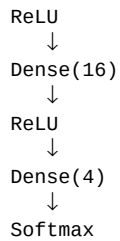
11. We built the neural network without a CNN

You specifically said:

Don't use the CNN part; let's develop only a neural network classifier.

So we created a very small TensorFlow MLP:

```
Input
64
  ↓
Dense(32)
  ↓
```



In shorthand:

$64 \rightarrow 32 \rightarrow 16 \rightarrow 4$

This network has only:

2676 trainable parameters

which makes it very attractive for FPGA implementation.

12. The FP32 model achieved 98.3%

The model achieved:

Test accuracy:
0.9830096960067749

or:

98.30%

with:

Test loss:
0.09787

This is a major improvement over the earlier grayscale system.

13. We inspected the confusion matrix

Your test set has 412 images.

The confusion matrix was:

	Predicted			
	FC	P	SD	SC
Fuelcap	55	0	6	1
Pliers	0	126	0	0
Screwdrivers	0	0	97	0
Sodacan	0	0	0	127

Therefore:

Fuelcap

$55/62=88.71\%$

Pliers

$126/126=100\%$

Screwdrivers

$97/97=100\%$

Sodacan

$127/127=100\%$

Overall:

$\boxed{98.30\%}$

The remaining errors are almost entirely in **fuelcap vs screwdriver**.

That's useful information for later feature analysis, but **we don't need to redesign the model yet**.

14. We tested the INT8 representation

This was one of the most important experiments.

We took the INT8 features:

```
FP32 standardized
  ↓
INT8
  ↓
dequantization
  ↓
FP32 approximation
```

and fed them into the **same trained TensorFlow model**.

The result was:

FP32 accuracy:
98.3009696%

INT8/dequantized accuracy:
98.3009696%

So:

$\boxed{\text{No observed accuracy loss}}$

That's excellent.

15. We also measured quantization error

We obtained:

Mean absolute error:
0.007999

Maximum absolute error:
0.074816

RMSE:
0.009196

So the INT8 representation is preserving the 64-feature vector very well.

This gives us strong evidence that **INT8 is a reasonable representation for the FPGA input.**

16. We identified an important distinction

We have currently quantized the **features**, not the neural-network weights.

We've demonstrated:

$\boxed{64 \text{ FP32 features} \rightarrow 64 \text{ INT8 features}}$

without measurable accuracy loss.

But we have **not yet converted the MLP weights/biases to an integer hardware representation.**

That's the next major hardware step.

17. We discussed the eventual FPGA arithmetic

The intended FPGA calculation will be something like:

$$y = \sum_i x_i w_i + b$$

with INT8 inputs and potentially INT8 weights.

Conceptually:

```
INT8 input
  x
INT8 weight
  ↓
INT16 product
  ↓
INT32 accumulator
  ↓
bias
  ↓
ReLU
  ↓
next layer
```

We should use a wider accumulator because adding many products can overflow INT16.

But we haven't implemented this yet.

18. We noticed the `.npy` issue

The feature arrays existed in Jupyter memory:

```
X_train
X_val
X_test

y_train
y_val
y_test
```

but we hadn't actually saved them as .npy files.

We then prepared code to save:

```
X_train.npy
y_train.npy

X_val.npy
y_val.npy

X_test.npy
y_test.npy
```

and also:

```
X_train_std.npy
X_val_std.npy
X_test_std.npy

X_train_int8.npy
X_val_int8.npy
X_test_int8.npy
```

So we now have a clean way of preserving the extracted dataset.

19. We also identified the important metadata files

We want to preserve:

```
classes.json
feature_names.json
scaler.npz
```

`classes.json`

Maps:

```
0 → fuelcap
1 → pliers
2 → screwdrivers
3 → sodacan
```

`feature\_names.json`

Contains the names of the 64 engineered features.

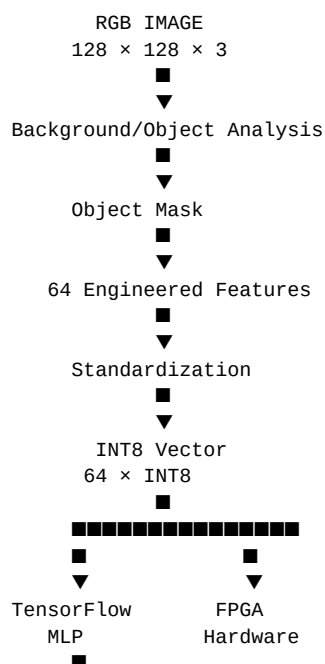
`scaler.npz`

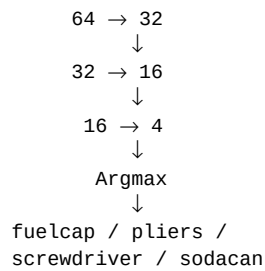
Contains the training-set preprocessing parameters required to reproduce the standardization/quantization.

`scaler.npz` is **particularly important** because the FPGA/embedded software must eventually perform the same preprocessing transformation.

Where we are now

The project currently looks like this:





Current measured performance

`\boxed{98.30\%\text{ test accuracy}}`

and:

`\boxed{98.30\%\text{ after INT8 feature quantization}}`

That's a very strong point to proceed from.

What remains to be done

I would divide the remaining work into four stages:

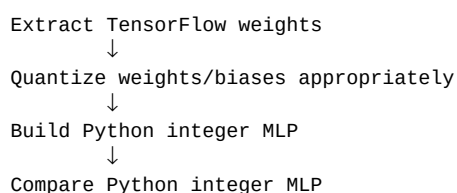
Stage 1 — Finish the software model

Current stage

- [x] RGB preprocessing
- [x] Object segmentation
- [x] 64-feature extraction
- [x] Dataset generation
- [x] Feature standardization
- [x] INT8 feature quantization
- [x] TensorFlow MLP
- [x] Confusion matrix
- [x] INT8 feature validation

Stage 2 — Make an exact integer reference

Next:



against TensorFlow model

This gives us a **golden reference** before touching Vitis HLS.

Stage 3 — FPGA implementation

Then:

```
INT8 features
  ↓
Dense 64→32
  ↓
ReLU
  ↓
Dense 32→16
  ↓
ReLU
  ↓
Dense 16→4
  ↓
Argmax
```

in **Vitis HLS**, followed by:

```
HLS IP
  ↓
Vivado IP Integrator
  ↓
Zynq-7020 / PYNQ-Z2
  ↓
AXI
```

Stage 4 — Hardware validation

Finally compare:

```
Python prediction
      vs
FPGA prediction
```

for the same test vectors.

Then measure:

- FPGA resource utilization
- DSP usage
- LUTs
- BRAM
- latency
- clock frequency
- throughput
- power, if available
- classification accuracy

The key achievement so far

The biggest thing we've established is that **you don't actually need to send the 128×128 RGB image into the FPGA.**

You can perform the computationally heavier image/feature processing on the Zynq PS and reduce:

49,152\text{ RGB pixel values}

to:

\boxed{64\text{ INT8 features}}

and then use the FPGA fabric for the tiny neural-network inference.

That gives you a clear, defensible **PS + PL hardware/software partition** for your PYNQ-Z2 project.